

实验二 组合部件的实现

班级 _____ 姓名 _____ 学号 _____

一、实验目的

1. 学习运用硬件描述语言 Verilog 描述组合电路。
2. 学习使用 Quartus II 开发环境。
3. 理解多路复用器、译码器、运算单元的工作原理及实现方式。

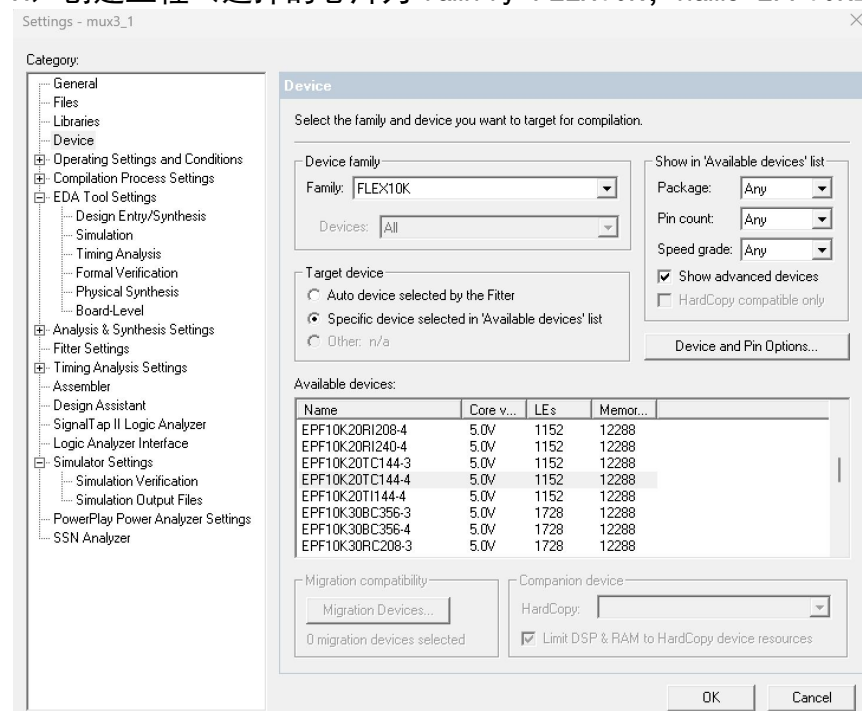
二、实验内容

1. 用 Verilog HDL 实现 8 重 2-1 多路复用器、8 重 3-1 多路复用器。
2. 用 Verilog HDL 实现指令译码器。
3. 用 Verilog HDL 实现运算单元 AU。

三、实验过程

1、8 重 3-1 多路复用器

A) 创建工程（选择的芯片为 family=FLEX10K; name=EPF10K20TC144-4）



B) 编写源代码

```
1 module mux3_1(  
2     input [7:0] a,  
3     input [7:0] b,  
4     input [7:0] c,  
5     input [1:0] s,  
6     output reg [7:0] y  
7 );  
8  
9 always @(*) begin  
10     case(s)  
11         2'b00: y = a;  
12         2'b01: y = b;  
13         2'b10: y = c;  
14         default: y = a;  
15     endcase  
16 end  
17  
18 endmodule
```

C) 编译与调试（包含编译调试过程中的错误、警告信息以及资源消耗）

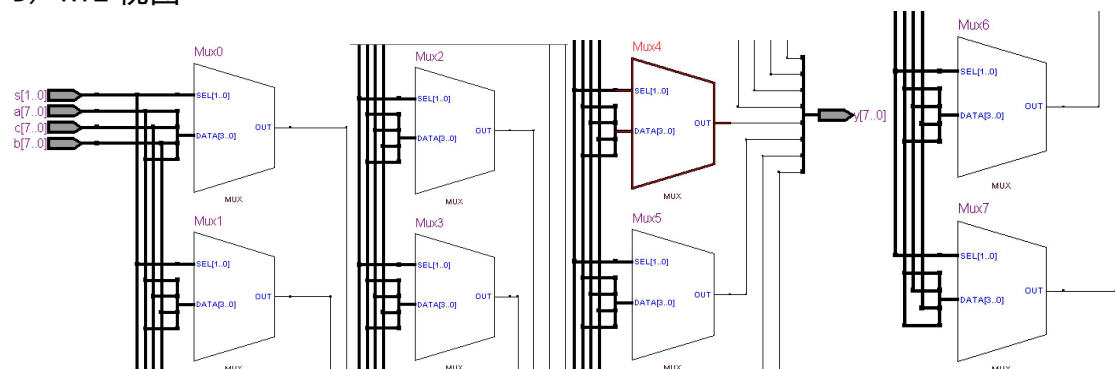
Quartus II

 Full Compilation was successful

确定

Flow Status	Successful - Sat Nov 29 12:57:46 2025
Quartus II Version	9.0 Build 184 04/29/2009 SP 1 SJ Web Edition
Revision Name	mux3_1
Top-level Entity Name	mux3_1
Family	FLEX10K
Device	EPF10K20TC144-4
Timing Models	Final
Met timing requirements	Yes
Total logic elements	16 / 1,152 (1 %)
Total pins	34 / 102 (33 %)
Total memory bits	0 / 12,288 (0 %)

D) RTL 视图

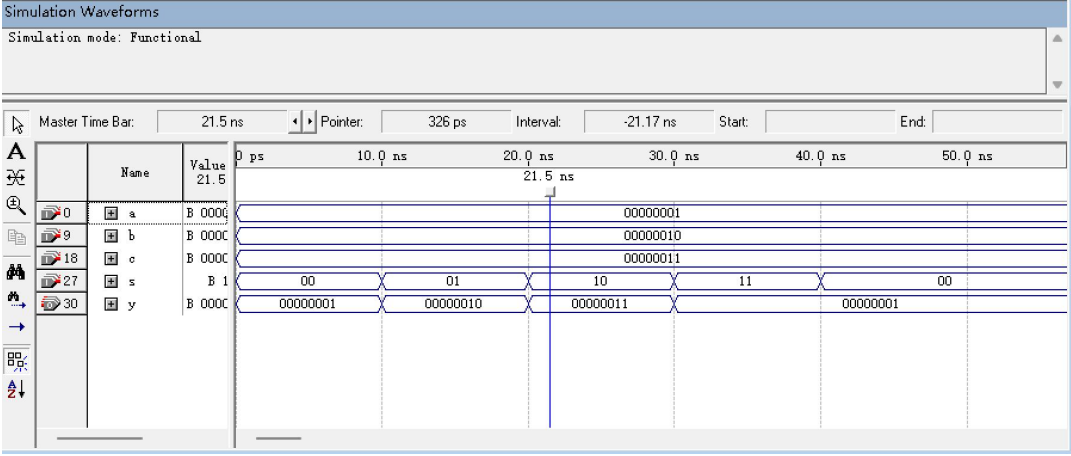


视图分析及结论:

视图分析: 通过观察 RTL 视图可知, 输入信号包括 8 位数字信号 a [7..0]、b [7..0]、c [7..0] 和 2 位控制信号 s [1..0], 输出信号为 8 位数字信号 y [7..0]; 电路包含 8 个独立的 Mux 模块 (Mux0-Mux7), 每个模块均配有 SEL [1..0] 控制端和 DATA [3..0] 数据端, 控制信号 s 通过 SEL 端口同步控制所有 Mux 模块, 实现对 a、b、c 信号的选择, 各输入、输出端口通过导线直接连接。

结论：Verilog 语言中简洁的代码对应实际硬件的内部结构较为复杂，一个简单的多路选择功能需要多个模块协同工作，通过多重门电路的组合实现。

E) 功能仿真波形

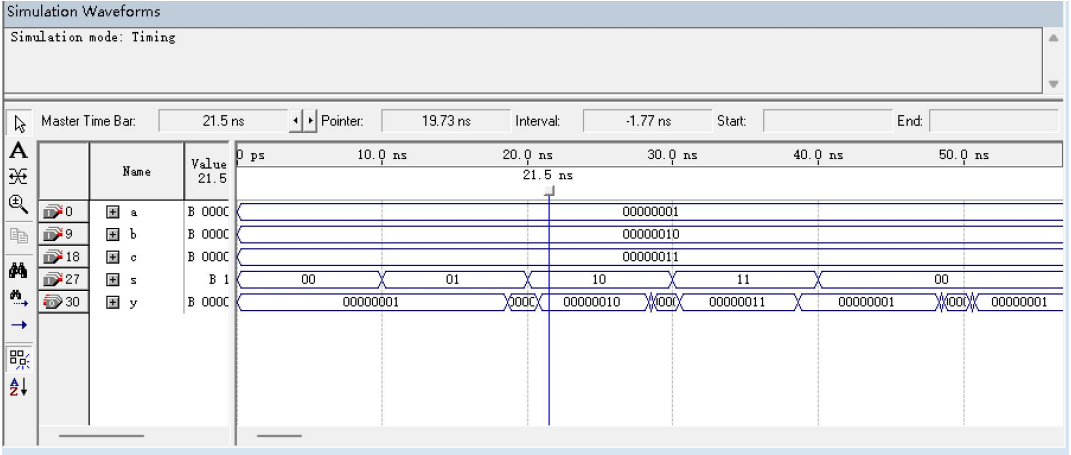


结果分析及结论:

结果分析：功能仿真是忽略器件延时和布线延时的理想仿真，用于验证逻辑功能。由仿真波形可见，输入信号状态变化时，输出信号实时响应无延迟，完全符合设计真值表：当 $s=2'b00$ 或 $2'b11$ （默认）时， $y=a$ ； $s=2'b01$ 时， $y=b$ ； $s=2'b10$ 时， $y=c$ 。

结论：功能仿真操作简便，能快速验证电路逻辑功能的正确性，但因忽略延时影响，仿真结果与实际硬件运行结果存在一定误差。

F) 时序仿真波形



结果分析及结论:

结果分析：时序仿真是布线后接近真实器件运行的仿真，包含器件和布线的延时信息。由波形可见，输入信号状态改变后，输出信号并未即时响应，存在明显延迟，且输入状态切换时可能出现电路“冒险”现象，导致输出结果短暂偏离预期。

结论：1、时序仿真可验证程序在目标器件中的时序关系，考虑延时后结果更接近实际，但

需兼顾的因素较多，操作复杂度高于功能仿真；2、时序仿真不仅反映输入输出的逻辑关系，还计算了延时信息，是贴近实际系统的仿真结果，但延时数值为仿真软件估算值。

G) 时序分析

操作方法是：编译后，在 compilation report 中选择【timing analysis】-【summary】和【tpd】

Timing Analyzer Summary										
	Type	Slack	Required Time	Actual Time	From	To	From Clock	To Clock	Failed Paths	
1	Worst-case tpd	N/A	None	10.752 ns	s[0]	y[0]	--	--	0	
2	Total number of failed paths								0	
tpd										
	Slack	Required P2P Time	Actual P2P Time	From	To	25	N/A	None	8.495 ns	s[1] y[2]
						26	N/A	None	8.480 ns	c[6] y[6]
1	N/A	None	10.752 ns	s[0]	y[0]	27	N/A	None	8.320 ns	s[1] y[1]
2	N/A	None	10.430 ns	s[1]	y[0]	28	N/A	None	8.267 ns	b[5] y[5]
3	N/A	None	10.076 ns	s[0]	y[7]	29	N/A	None	8.092 ns	c[5] y[5]
4	N/A	None	9.896 ns	c[0]	y[0]	30	N/A	None	8.071 ns	b[4] y[4]
5	N/A	None	9.824 ns	s[0]	y[3]	31	N/A	None	8.064 ns	a[4] y[4]
6	N/A	None	9.752 ns	s[1]	y[7]	32	N/A	None	8.045 ns	b[6] y[6]
7	N/A	None	9.738 ns	a[0]	y[0]	33	N/A	None	8.031 ns	a[6] y[6]
8	N/A	None	9.502 ns	s[1]	y[3]	34	N/A	None	7.855 ns	c[4] y[4]
9	N/A	None	9.464 ns	s[0]	y[5]	35	N/A	None	7.771 ns	a[2] y[2]
10	N/A	None	9.391 ns	b[0]	y[0]	36	N/A	None	7.696 ns	a[1] y[1]
11	N/A	None	9.161 ns	b[7]	y[7]	37	N/A	None	7.688 ns	c[2] y[2]
12	N/A	None	9.159 ns	s[0]	y[6]	38	N/A	None	7.663 ns	b[2] y[2]
13	N/A	None	9.141 ns	s[1]	y[5]	39	N/A	None	7.254 ns	c[1] y[1]
14	N/A	None	9.080 ns	a[7]	y[7]	40	N/A	None	7.060 ns	b[1] y[1]
15	N/A	None	9.078 ns	a[5]	y[5]					
16	N/A	None	9.037 ns	s[0]	y[4]					
17	N/A	None	8.895 ns	c[7]	y[7]					
18	N/A	None	8.835 ns	s[1]	y[6]					
19	N/A	None	8.817 ns	s[0]	y[2]					
20	N/A	None	8.810 ns	a[3]	y[3]					
21	N/A	None	8.714 ns	s[1]	y[4]					
22	N/A	None	8.663 ns	b[3]	y[3]					
23	N/A	None	8.642 ns	s[0]	y[1]					
24	N/A	None	8.588 ns	c[3]	y[3]					

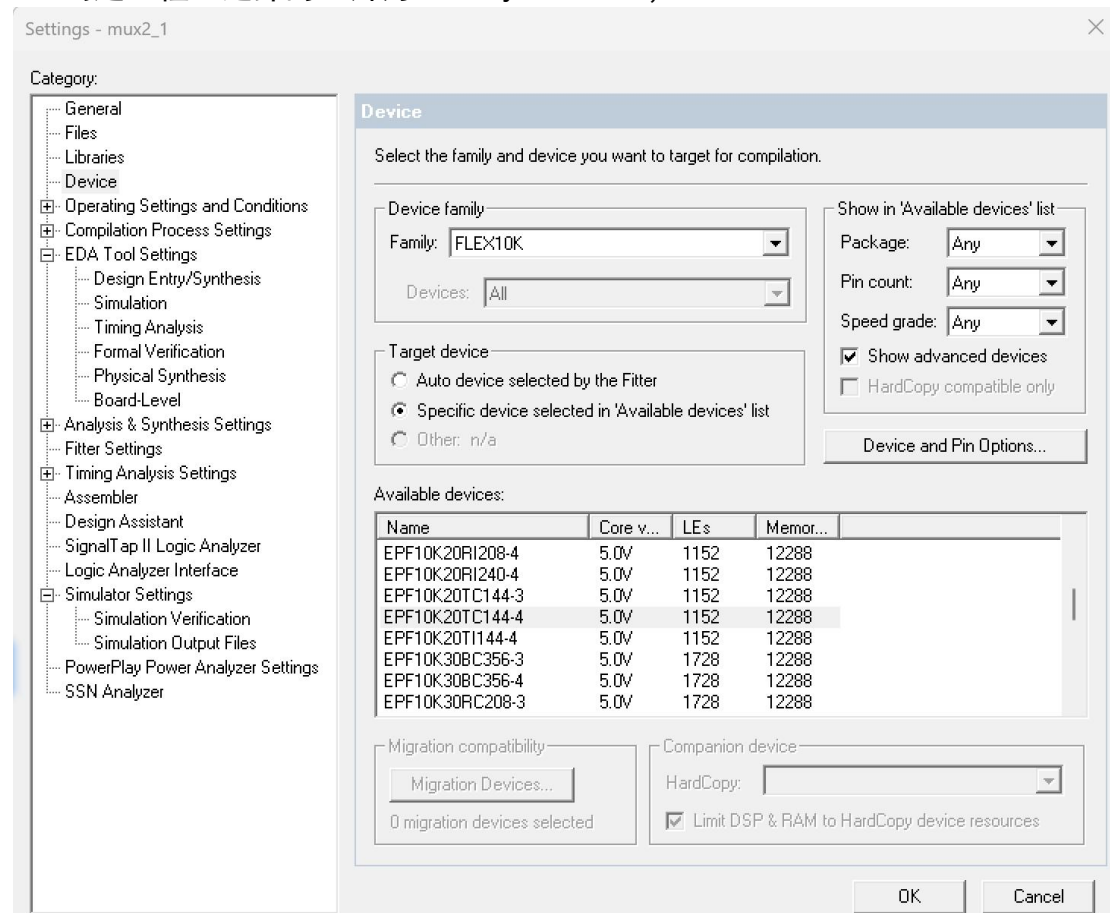
结果分析及结论：

结果分析：Timing Analyzer Summary 汇总了经典定时分析结果，报告了各定时特性的最坏情况延时，例如 s [0] 到 y [0] 的最坏 tpd（引脚到引脚延时）为 10.752ns；tpd 报告表详细列出了源节点与目标节点之间的具体延时数值，不同节点组合的延时存在差异。

结论：实际硬件电路中，各元器件的连接存在固有延时，且不同信号路径（源节点 - 目标节点）的延时数值各不相同，需通过时序分析验证电路是否满足时序要求。

2、8 重 2-1 多路复用器

A) 创建工程（选择的芯片为 family=FLEX10K；name=EPF10K20TC144-4）



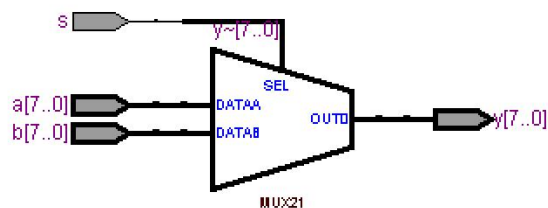
B) 编写源代码

```
1 module mux2_1(  
2     input [7:0] a,  
3     input [7:0] b,  
4     input s,  
5     output reg [7:0] y  
6 );  
7  
8 always @(*) begin  
9     y = s ? b : a;  
10 end  
11  
12 endmodule
```

C) 编译与调试（包含编译调试过程中的错误、警告信息以及资源消耗）

Type	Message
System (3) Processing (35) Extra Info Info (35) Warning	
Flow Status	Successful - Sat Nov 29 12:54:11 2025
Quartus II Version	9.0 Build 184 04/29/2009 SP 1 SJ Web Edition
Revision Name	mux2_1
Top-level Entity Name	mux2_1
Family	FLEX10K
Device	EPF10K20TC144-4
Timing Models	Final
Met timing requirements	Yes
Total logic elements	8 / 1,152 (< 1 %)
Total pins	25 / 102 (25 %)
Total memory bits	0 / 12,288 (0 %)

D) RTL 视图

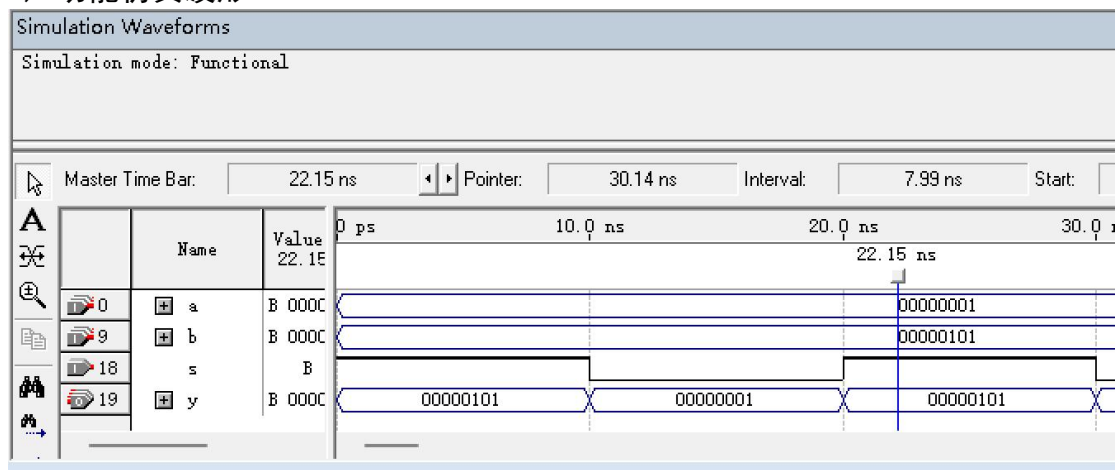


结果分析及结论：

视图分析：通过观察 RTL 视图可知，输入信号包括 8 位数字信号 $a[7..0]$ 、 $b[7..0]$ 和 1 位控制信号 s ，输出信号为 8 位数字信号 $y[7..0]$ ；电路核心为 MUX21 模块，控制信号 s 通过 SEL 端口控制模块选择输入信号 a 或 b ，各输入、输出端口通过导线直接连接。

结论：Verilog 语言中简洁的代码对应实际硬件的内部结构较为复杂，一个简单的二路选择功能需要通过特定功能模块及门电路的组合实现。

E) 功能仿真波形

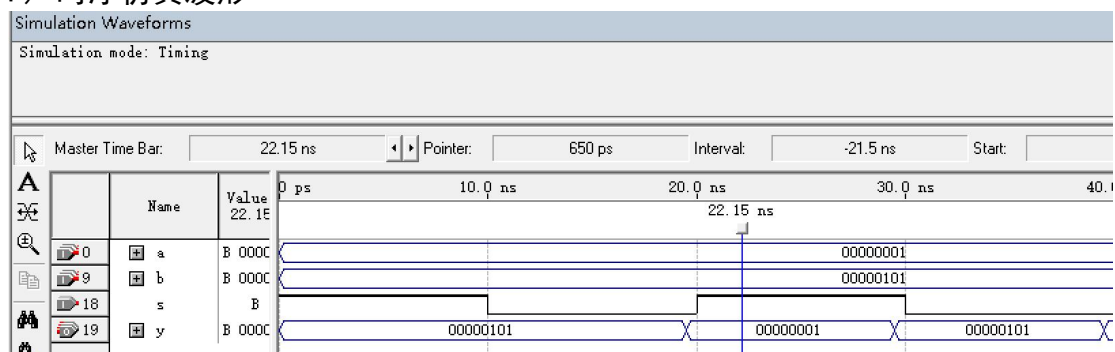


结果分析及结论:

结果分析：功能仿真是忽略器件延时和布线延时的理想仿真，用于验证逻辑功能。由仿真波形可见，输入信号状态变化时，输出信号实时响应无延迟，完全符合设计真值表：当 $s=1'b0$ 时， $y=a$ ； $s=1'b1$ 时， $y=b$ 。

结论：功能仿真操作简便，能快速验证电路逻辑功能的正确性，但因忽略延时影响，仿真结果与实际硬件运行结果存在一定误差。

F) 时序仿真波形



结果分析及结论:

结果分析：时序仿真是布线后接近真实器件运行的仿真，包含器件和布线的延时信息。由波形可见，输入信号状态改变后，输出信号并未即时响应，存在明显延迟，且输入状态切换时可能出现电路“冒险”现象，导致输出结果短暂偏离预期。

结论：1、时序仿真可验证程序在目标器件中的时序关系，考虑延时后结果更接近实际，但需兼顾的因素较多，操作复杂度高于功能仿真；2、时序仿真不仅反映输入输出的逻辑关系，还计算了延时信息，是贴近实际系统的仿真结果，但延时数值为仿真软件估算值。

G) 时序分析

Timing Analyzer Summary										
	Type	Slack	Required Time	Actual Time	From	To	From Clock	To Clock	Failed Paths	
1	Worst-case tpd	N/A	None	9.726 ns	s	y[7]	--	--	0	
2	Total number of failed paths								0	
tpd										
	Slack	Required P2P Time	Actual P2P Time	From	To					
1	N/A	None	9.726 ns	s	y[7]	13	N/A	None	7.893 ns	a[0] y[0]
2	N/A	None	9.514 ns	s	y[6]	14	N/A	None	7.873 ns	b[1] y[1]
3	N/A	None	9.478 ns	s	y[2]	15	N/A	None	7.713 ns	s y[5]
4	N/A	None	9.414 ns	a[7]	y[7]	16	N/A	None	7.692 ns	b[0] y[0]
5	N/A	None	9.350 ns	a[6]	y[6]	17	N/A	None	7.519 ns	s y[1]
6	N/A	None	9.298 ns	b[7]	y[7]	18	N/A	None	7.489 ns	s y[4]
7	N/A	None	9.050 ns	b[6]	y[6]	19	N/A	None	7.482 ns	b[4] y[4]
8	N/A	None	9.037 ns	a[2]	y[2]	20	N/A	None	7.469 ns	a[3] y[3]
9	N/A	None	8.955 ns	b[2]	y[2]	21	N/A	None	7.424 ns	b[3] y[3]
10	N/A	None	8.246 ns	b[5]	y[5]	22	N/A	None	7.273 ns	a[1] y[1]
11	N/A	None	8.170 ns	s	y[0]	23	N/A	None	7.263 ns	a[5] y[5]
12	N/A	None	7.899 ns	s	y[3]	24	N/A	None	7.004 ns	a[4] y[4]

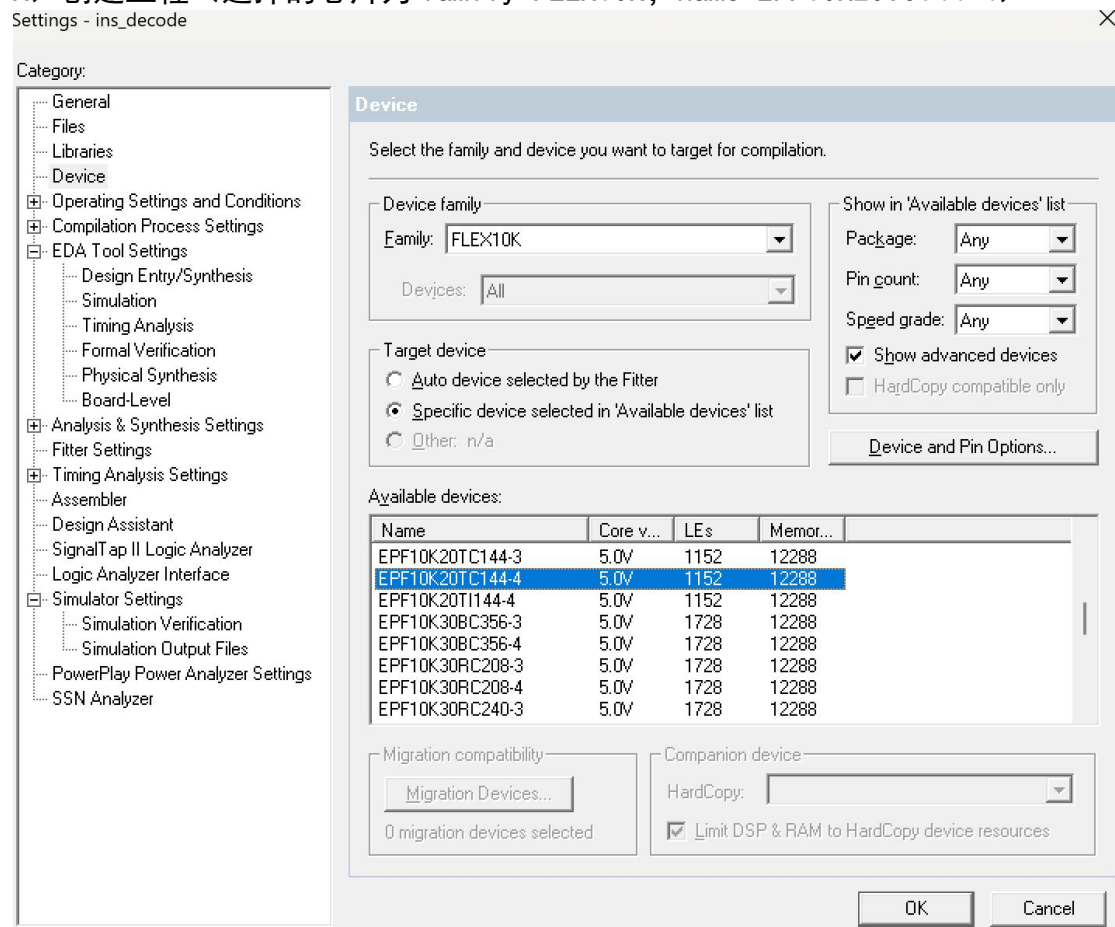
结果分析及结论:

结果分析: Timing Analyzer Summary 汇总了经典定时分析结果, 报告了最坏情况 tpd 为 9.726ns (如 s 到 y[7] 的延时); tpd 报告表详细列出了源节点与目标节点之间的具体延时数值, 不同节点组合的延时存在差异。

结论: 实际硬件电路中, 各元器件的连接存在固有延时, 且不同信号路径 (源节点 - 目标节点) 的延时数值各不相同, 需通过时序分析验证电路是否满足时序要求。

3、指令译码器

A) 创建工程（选择的芯片为 family=FLEX10K；name=EPF10K20TC144-4）



B) 编写源代码

```
1 module ins_decode (  
2     input en,  
3     input [3:0] ir,  
4     output reg mova,  
5     output reg movb,  
6     output reg movc,  
7     output reg movd,  
8     output reg add,  
9     output reg sub,  
10    output reg jmp,  
11    output reg jg,  
12    output reg in1,  
13    output reg out1,  
14    output reg movi,  
15    output reg halt  
16 );  
17  
18 always @(*) begin  
19     {mova, movb, movc, movd, add, sub, jmp, jg, in1, out1, movi, halt} = 12'b000000000000;  
20     if(en) begin  
21         case(ir)  
22             4'b0100: mova = 1'b1;  
23             4'b0101: movb = 1'b1;  
24             4'b0110: movc = 1'b1;  
25             4'b0111: movd = 1'b1;  
26             4'b1000: add = 1'b1;  
27             4'b1001: sub = 1'b1;  
28             4'b1010: jmp = 1'b1;  
29             4'b1011: jg = 1'b1;  
30             4'b1100: in1 = 1'b1;  
31             4'b1101: out1 = 1'b1;  
32             4'b1110: movi = 1'b1;  
33             4'b1111: halt = 1'b1;  
34             default: ;  
35         endcase  
36     end  
37 end  
38  
39 endmodule
```

D) 编译与调试（包含编译调试过程中的错误、警告信息以及资源消耗）

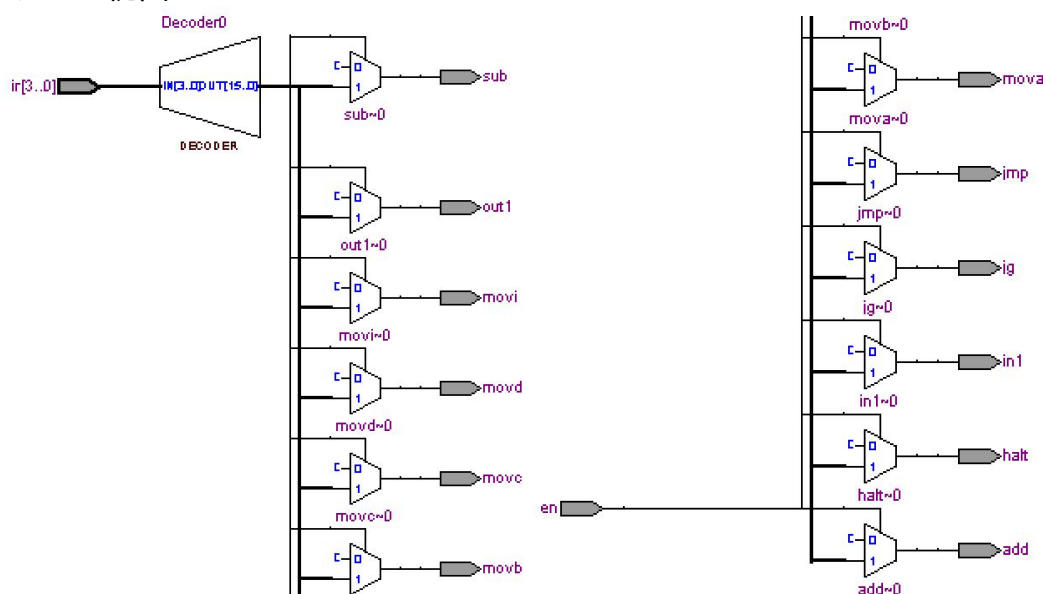
Quartus II

 Full Compilation was successful

确定

Flow Status	Successful - Sat Nov 29 12:59:22 2025
Quartus II Version	9.0 Build 184 04/29/2009 SP 1 SJ Web Edition
Revision Name	ins_decode
Top-level Entity Name	ins_decode
Family	FLEX10K
Device	EPF10K20TC144-4
Timing Models	Final
Met timing requirements	Yes
Total logic elements	24 / 1,152 (2 %)
Total pins	17 / 102 (17 %)
Total memory bits	0 / 12,288 (0 %)

D) RTL 视图

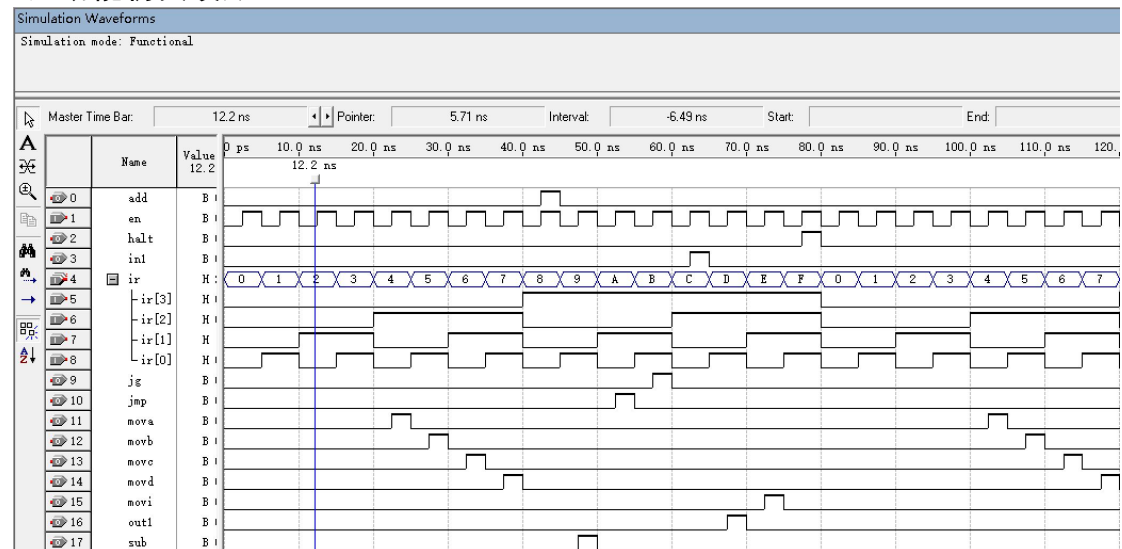


视图分析及结论：

视图分析：通过观察 RTL 视图可知，输入信号包括 4 位指令码 `ir[3..0]` 和使能信号 `en`，输出信号为 `movv`、`movb`、`movc` 等 12 个控制信号；电路核心为 Decoder 模块，`ir[3..0]` 作为译码输入，`en` 控制译码功能使能，各输出信号通过译码逻辑与输入指令码对应，输入与输出端口通过导线直接连接。

结论：Verilog 简洁的译码逻辑对应硬件上的译码器模块，需通过组合逻辑将门电路协同工作，实现指令码到控制信号的精准映射。

E) 功能仿真波形



结果分析及结论:

结果分析：功能仿真忽略延时，验证逻辑正确性。由波形可见，当 $en=1$ （使能有效）时，不同 ir 输入对应不同输出： $ir=4'b0100$ 时 $movb=1$ ， $ir=4'b1000$ 时 $add=1$ ， $ir=4'b1111$ 时 $halt=1$ ，完全符合代码中 `case` 语句的逻辑映射； $en=0$ 时所有输出均为 0。

结论：功能仿真快速验证了译码逻辑的正确性，操作简便，但未考虑硬件延时，与实际运行存在细微差异。

F) 时序仿真波形



结果分析及结论:

结果分析：时序仿真包含器件和布线延时。由波形可见， en 和 ir 状态改变后，输出控制信

号存在短暂延迟，输入切换时可能出现轻微“冒险”现象，导致输出信号短暂不稳定，但最终能稳定映射至正确状态。

结论：时序仿真更贴近硬件实际运行情况，可验证时序稳定性，虽操作较复杂，但能提前发现延迟带来的潜在问题，延时数值为软件估算值。

G) 时序分析

Timing Analyzer Summary														
	Type					Slack	Required Time	Actual Time	From	To	From Clock	To Clock	Failed Paths	
1	Worst-case tpd					N/A	None	9.794 ns	ir[1]	sub	--	--	0	
2	Total number of failed paths												0	
tpd						20	N/A	None	9.234 ns	ir[2]	add			
	Slack	Required P2P Time	Actual P2P Time	From	To	21	N/A	None	9.227 ns	ir[1]	movd	41	N/A	None
						22	N/A	None	9.161 ns	ir[1]	movb	42	N/A	None
1	N/A	None	9.794 ns	ir[1]	sub	23	N/A	None	9.154 ns	ir[0]	jmp	43	N/A	None
2	N/A	None	9.793 ns	ir[1]	out1	24	N/A	None	9.131 ns	ir[1]	movb	44	N/A	None
3	N/A	None	9.667 ns	ir[1]	movi	25	N/A	None	9.123 ns	ir[0]	movd	45	N/A	None
4	N/A	None	9.653 ns	ir[0]	sub	26	N/A	None	9.101 ns	ir[1]	ig	46	N/A	None
5	N/A	None	9.651 ns	ir[0]	out1	27	N/A	None	9.099 ns	ir[1]	in1	47	N/A	None
6	N/A	None	9.615 ns	ir[1]	add	28	N/A	None	9.098 ns	ir[1]	halt	48	N/A	None
7	N/A	None	9.569 ns	ir[0]	movi	29	N/A	None	9.034 ns	ir[0]	movb	49	N/A	None
8	N/A	None	9.512 ns	ir[0]	add	30	N/A	None	9.026 ns	ir[0]	movb	50	N/A	None
9	N/A	None	9.408 ns	ir[3]	add	31	N/A	None	9.025 ns	en	add	51	N/A	None
10	N/A	None	9.375 ns	ir[2]	sub	32	N/A	None	9.020 ns	ir[3]	movd	52	N/A	None
11	N/A	None	9.372 ns	ir[2]	out1	33	N/A	None	8.997 ns	ir[0]	halt	53	N/A	None
12	N/A	None	9.339 ns	ir[3]	sub	34	N/A	None	8.995 ns	ir[0]	in1	54	N/A	None
13	N/A	None	9.338 ns	ir[3]	out1	35	N/A	None	8.991 ns	ir[1]	movc	55	N/A	None
14	N/A	None	9.306 ns	ir[2]	movi	36	N/A	None	8.960 ns	ir[0]	ig	56	N/A	None
15	N/A	None	9.301 ns	en	sub	37	N/A	None	8.897 ns	ir[3]	halt	57	N/A	None
16	N/A	None	9.299 ns	en	out1	38	N/A	None	8.892 ns	ir[3]	in1	58	N/A	None
17	N/A	None	9.261 ns	en	movi	39	N/A	None	8.889 ns	ir[0]	movc	59	N/A	None
18	N/A	None	9.259 ns	ir[3]	movi	40	N/A	None	8.877 ns	ir[2]	jmp	60	N/A	None
19	N/A	None	9.256 ns	ir[1]	jmp									

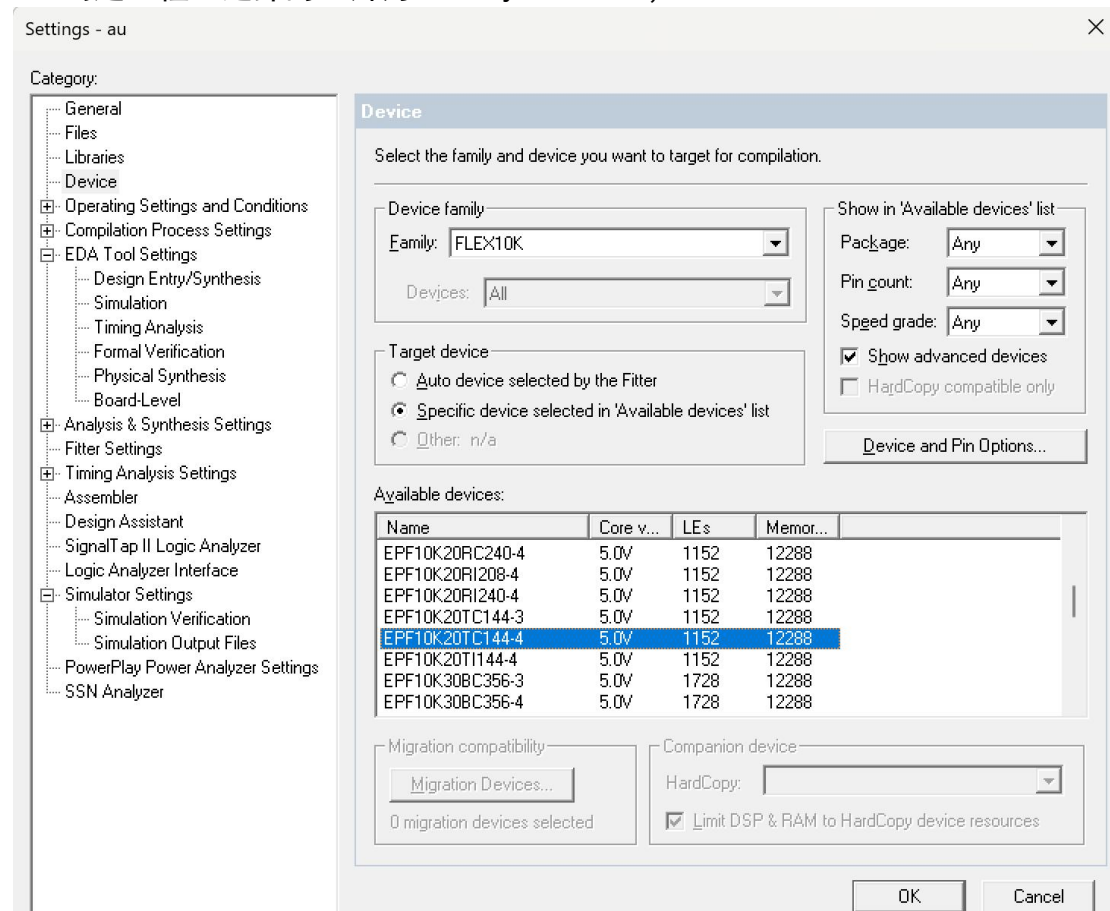
结果分析及结论：

结果分析：Timing Analyzer Summary 显示，最坏情况 tpd 为 9.794ns（ir[1] 到 sub 的路径）；tpd 报告表显示，不同输入输出路径的延时存在差异，如 ir[1] 到 out1 的延时为 9.793ns，ir[0] 到 sub 的延时为 9.653ns。

结论：硬件电路中不同信号路径的固有延时不同，需通过时序分析确认关键路径延时是否满足设计要求，本实验所有路径均无失败，时序达标。

4、算术单元 AU

A) 创建工程（选择的芯片为 family=FLEX10K； name=EPF10K20TC144-4）



B) 编写源代码

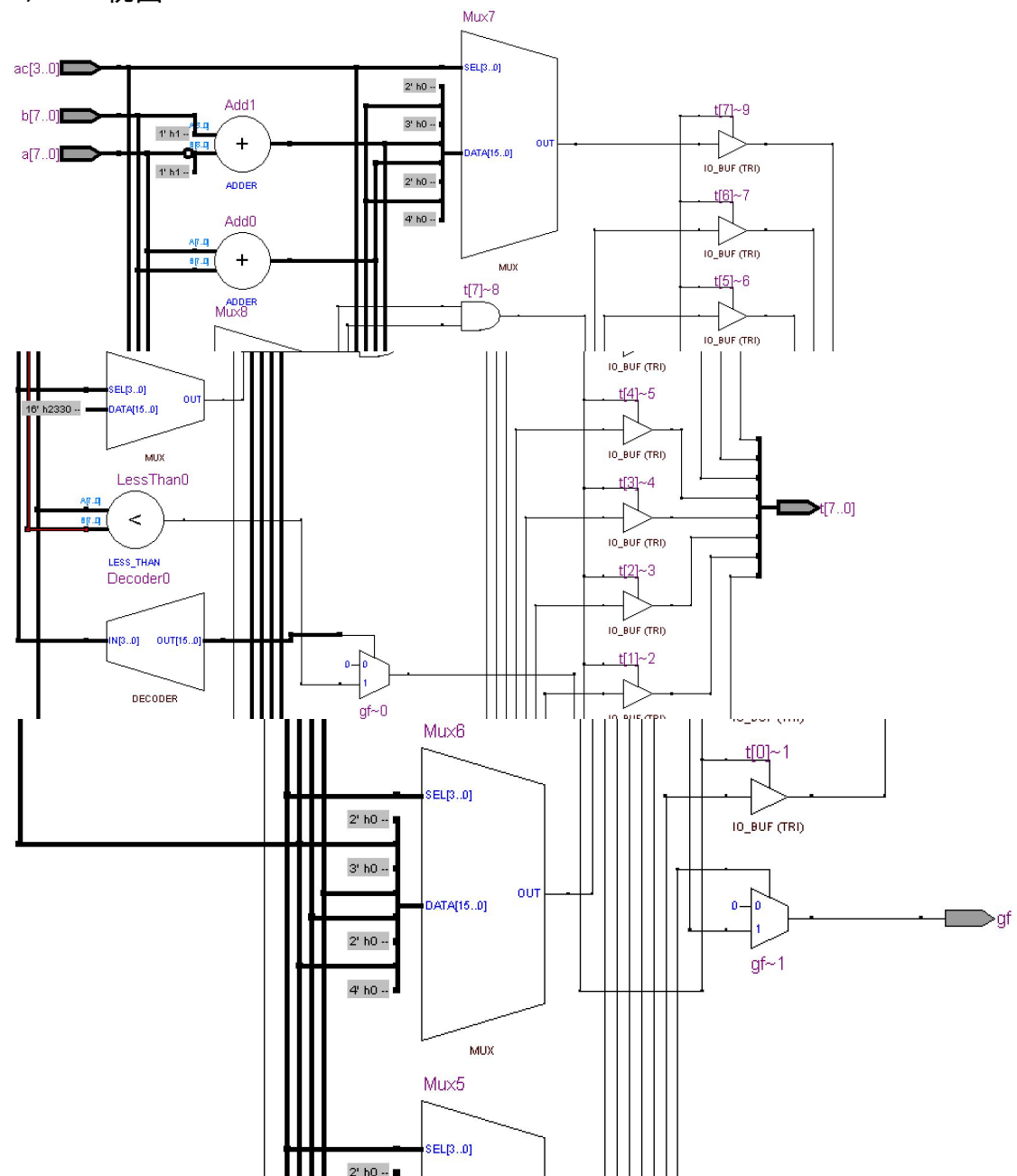
```
1 module au(  
2     input    au_en,  
3     input [3:0] ac,  
4     input [7:0] a,  
5     input [7:0] b,  
6     output reg [7:0] t,  
7     output reg gf  
8 );  
9  
10 always @(*) begin  
11     if (au_en) begin  
12         case (ac)  
13             4'b1000: begin  
14                 t = a + b;  
15                 gf = 1'b0;  
16             end  
17             4'b1001: begin  
18                 t = b - a;  
19                 gf = (b > a) ? 1'b1 : 1'b0;  
20  
21                 end  
22                 4'b0100,  
23                 4'b0101,  
24                 4'b1101: begin  
25                     t = a;  
26                     gf = 1'b0;  
27                 end  
28                 default: begin  
29                     t = 8'hZZ;  
30                     gf = 1'b0;  
31                 end  
32             endcase  
33         end else begin  
34             t = 8'hZZ;  
35             gf = 1'b0;  
36         end  
37     end  
38 endmodule
```

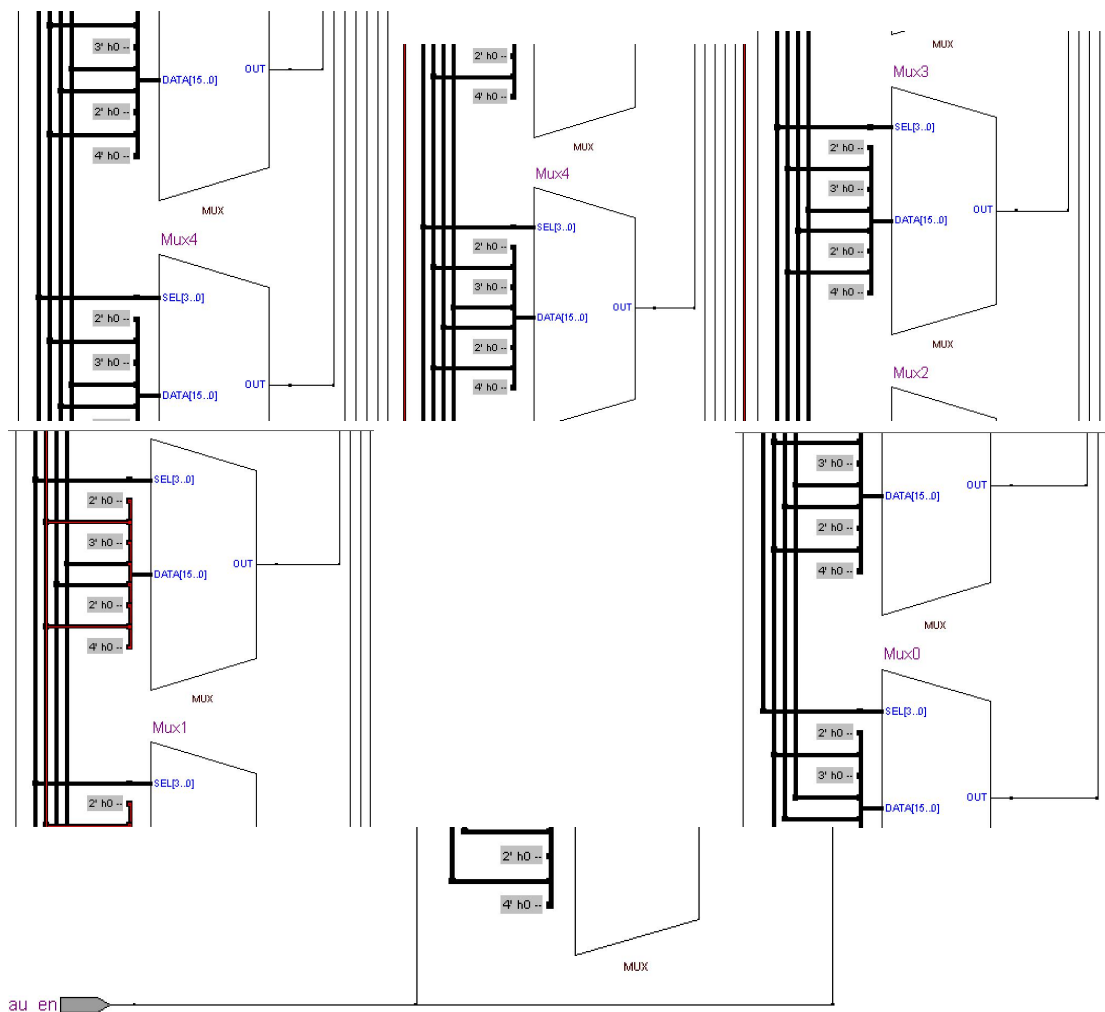
D) 编译与调试（包含编译调试过程中的错误、警告信息以及资源消耗）

Type Message

System [3]	Processing [49]	Extra Info	Info [49]	Warning
Flow Status	Successful - Sat Nov 29 11:50:31 2025			
Quartus II Version	9.0 Build 184 04/29/2009 SP 1 SJ Web Edition			
Revision Name	au			
Top-level Entity Name	au			
Family	FLEX10K			
Device	EPF10K20TC144-4			
Timing Models	Final			
Met timing requirements	Yes			
Total logic elements	45 / 1,152 (4 %)			
Total pins	30 / 102 (29 %)			
Total memory bits	0 / 12,288 (0 %)			

D) RTL 视图



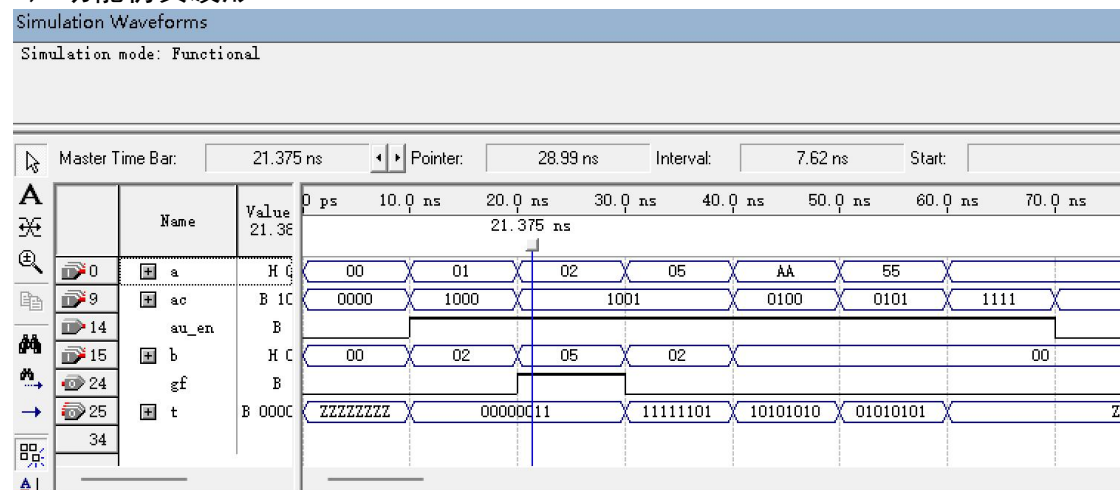


结果分析及结论:

视图分析: 通过观察 RTL 视图可知, 输入信号包括使能信号 `au_en`、4 位运算控制信号 `ac`、8 位操作数 `a[7..0]` 和 `b[7..0]`, 输出信号为 8 位运算结果 `t[7..0]` 和标志位 `gf`; 电路包含加法器 (Add0、Add1)、多路选择器 (Mux1-Mux8) 和译码器模块, `ac` 控制运算类型, `au_en` 控制模块使能, 运算逻辑通过加法器和组合逻辑实现。

结论: 算术单元的硬件结构复杂, Verilog 代码中的运算逻辑需通过加法器、多路选择器等器件协同, 实现加法、减法及数据传输等多种功能。

E) 功能仿真波形

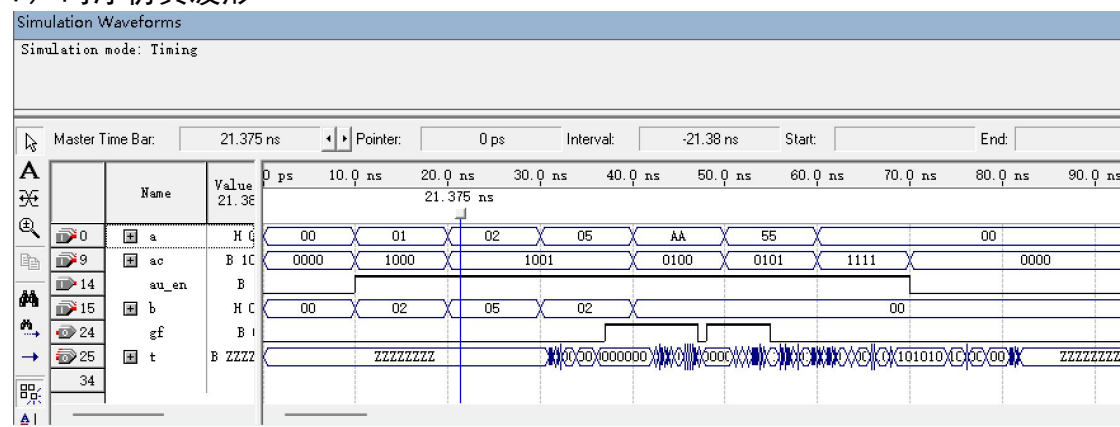


结果分析及结论:

结果分析：功能仿真忽略延时，验证运算逻辑。由波形可见，当 $au_en=1$ （使能有效）时， $ac=4'b1000$ 时 $t=a+b$ （加法运算）， $ac=4'b1001$ 时 $t=b-a$ 且 $gf=(b>a)$ （减法运算）， $ac=4'b0100/0101/1101$ 时 $t=a$ （数据传输），完全符合代码逻辑； $au_en=0$ 时 $t=8'hZZ$ （高阻态）。

结论：功能仿真准确验证了 AU 的运算逻辑正确性，操作简便，可快速排查代码逻辑错误，但未反映硬件延时特性。

F) 时序仿真波形



结果分析及结论:

结果分析：时序仿真包含器件和布线延时。由波形可见， au_en 、 ac 或操作数 a 、 b 状态改变后，输出 t 和 gf 存在明显延迟，输入切换时可能出现“冒险”现象，导致结果短暂偏离预期，但稳定后与设计逻辑一致。

结论：时序仿真反映了 AU 的实际运行时序特性，考虑延时后结果更贴近硬件实际，虽操作复杂，但能为时序优化提供依据，延时为软件估算值。

G) 时序分析

Timing Analyzer Summary										
	Type	Slack	Required Time	Actual Time	From	To	From Clock	To Clock	Failed Paths	
1	Worst-case tpd	N/A	None	29.400 ns	b[2]	gf	--	--	0	
2	Total number of failed paths								0	

tpd						tpd						tpd					
	Slack	Required P2P Time	Actual P2P Time	From	To		Slack	Required P2P Time	Actual P2P Time	From	To		Slack	Required P2P Time	Actual P2P Time	From	To
1	N/A	None	29.400 ns	b[2]	gf	26	N/A	None	25.700 ns	b[0]	t[5]	51	N/A	None	23.900 ns	a[6]	t[7]
2	N/A	None	28.000 ns	b[3]	gf	27	N/A	None	25.500 ns	a[6]	t[6]	52	N/A	None	23.900 ns	a[0]	t[0]
3	N/A	None	27.900 ns	a[0]	gf	28	N/A	None	25.500 ns	b[1]	t[6]	53	N/A	None	23.800 ns	b[3]	t[7]
4	N/A	None	27.600 ns	b[0]	gf	29	N/A	None	25.500 ns	b[1]	t[4]	54	N/A	None	23.800 ns	b[5]	t[5]
5	N/A	None	27.300 ns	a[1]	gf	30	N/A	None	25.400 ns	b[3]	t[4]	55	N/A	None	23.800 ns	a[0]	t[3]
6	N/A	None	27.000 ns	b[1]	gf	31	N/A	None	25.300 ns	b[3]	t[6]	56	N/A	None	23.700 ns	a[1]	t[3]
7	N/A	None	26.700 ns	b[2]	t[4]	32	N/A	None	25.200 ns	b[6]	gf	57	N/A	None	23.700 ns	b[0]	t[0]
8	N/A	None	26.600 ns	b[2]	t[6]	33	N/A	None	25.200 ns	a[5]	t[5]	58	N/A	None	23.600 ns	a[2]	t[6]
9	N/A	None	26.600 ns	b[4]	t[4]	34	N/A	None	25.100 ns	b[2]	t[7]	59	N/A	None	23.600 ns	a[2]	t[4]
10	N/A	None	26.500 ns	b[4]	t[6]	35	N/A	None	25.100 ns	b[6]	t[6]	60	N/A	None	23.600 ns	b[0]	t[3]
11	N/A	None	26.500 ns	a[0]	t[4]	36	N/A	None	25.000 ns	b[4]	t[7]	61	N/A	None	23.500 ns	b[6]	t[7]
12	N/A	None	26.400 ns	a[0]	t[6]	37	N/A	None	24.900 ns	a[0]	t[7]	62	N/A	None	23.300 ns	a[3]	t[6]
13	N/A	None	26.400 ns	a[1]	t[6]	38	N/A	None	24.900 ns	a[1]	t[7]	63	N/A	None	23.300 ns	a[3]	t[4]
14	N/A	None	26.400 ns	a[1]	t[4]	39	N/A	None	24.900 ns	b[1]	t[5]	64	N/A	None	23.300 ns	a[4]	t[4]
15	N/A	None	26.300 ns	a[2]	gf	40	N/A	None	24.800 ns	b[3]	t[5]	65	N/A	None	23.300 ns	b[1]	t[2]
16	N/A	None	26.300 ns	a[3]	gf	41	N/A	None	24.700 ns	b[7]	gf	66	N/A	None	23.200 ns	a[4]	t[6]
17	N/A	None	26.300 ns	b[0]	t[4]	42	N/A	None	24.700 ns	b[0]	t[7]	67	N/A	None	23.200 ns	a[0]	t[1]
18	N/A	None	26.200 ns	b[0]	t[6]	43	N/A	None	24.600 ns	b[2]	t[2]	68	N/A	None	23.200 ns	a[1]	t[1]
19	N/A	None	26.100 ns	b[2]	t[5]	44	N/A	None	24.300 ns	b[5]	t[6]	69	N/A	None	23.000 ns	a[2]	t[5]
20	N/A	None	25.900 ns	b[4]	gf	45	N/A	None	24.300 ns	a[0]	t[2]	70	N/A	None	23.000 ns	b[0]	t[1]
21	N/A	None	25.900 ns	b[4]	t[5]	46	N/A	None	24.200 ns	a[5]	t[7]	71	N/A	None	22.800 ns	b[5]	t[7]
22	N/A	None	25.900 ns	a[0]	t[5]	47	N/A	None	24.200 ns	a[1]	t[2]	72	N/A	None	22.800 ns	b[3]	t[3]
23	N/A	None	25.800 ns	a[1]	t[5]	48	N/A	None	24.100 ns	b[0]	t[2]	73	N/A	None	22.800 ns	b[1]	t[3]
24	N/A	None	25.700 ns	a[6]	gf	49	N/A	None	24.000 ns	b[1]	t[7]	74	N/A	None	22.700 ns	a[3]	t[5]
25	N/A	None	25.700 ns	a[5]	t[6]	50	N/A	None	24.000 ns	b[2]	t[3]	75	N/A	None	22.600 ns	a[4]	gf

tpd						tpd					
	Slack	Required P2P Time	Actual P2P Time	From	To		Slack	Required P2P Time	Actual P2P Time	From	To
76	N/A	None	22.600 ns	a[4]	t[5]	101	N/A	None	20.900 ns	au_en	t[0]
77	N/A	None	22.500 ns	a[7]	gf	102	N/A	None	20.800 ns	a[5]	gf
78	N/A	None	22.500 ns	b[7]	t[7]	103	N/A	None	20.800 ns	au_en	t[4]
79	N/A	None	22.300 ns	b[1]	t[1]	104	N/A	None	20.800 ns	ac[1]	t[2]
80	N/A	None	22.100 ns	a[2]	t[7]	105	N/A	None	20.700 ns	a[3]	t[3]
81	N/A	None	22.000 ns	ac[3]	t[0]	106	N/A	None	20.400 ns	ac[0]	t[0]
82	N/A	None	21.900 ns	ac[3]	t[4]	107	N/A	None	20.400 ns	ac[2]	t[0]
83	N/A	None	21.800 ns	a[3]	t[7]	108	N/A	None	20.300 ns	au_en	t[5]
84	N/A	None	21.800 ns	ac[1]	t[0]	109	N/A	None	20.300 ns	ac[0]	t[4]
85	N/A	None	21.700 ns	a[4]	t[7]	110	N/A	None	20.300 ns	ac[2]	t[4]
86	N/A	None	21.700 ns	ac[1]	t[4]	111	N/A	None	20.100 ns	au_en	t[6]
87	N/A	None	21.500 ns	a[2]	t[2]	112	N/A	None	20.000 ns	au_en	t[7]
88	N/A	None	21.400 ns	ac[3]	t[5]	113	N/A	None	20.000 ns	au_en	t[3]
89	N/A	None	21.200 ns	ac[3]	t[6]	114	N/A	None	20.000 ns	au_en	t[1]
90	N/A	None	21.200 ns	ac[1]	t[5]	115	N/A	None	19.900 ns	au_en	t[2]
91	N/A	None	21.100 ns	ac[3]	t[7]	116	N/A	None	19.800 ns	ac[0]	t[5]
92	N/A	None	21.100 ns	ac[3]	t[3]	117	N/A	None	19.800 ns	ac[2]	t[5]
93	N/A	None	21.100 ns	ac[3]	t[1]	118	N/A	None	19.600 ns	ac[0]	t[6]
94	N/A	None	21.000 ns	ac[1]	t[6]	119	N/A	None	19.600 ns	ac[2]	t[6]
95	N/A	None	21.000 ns	ac[3]	t[2]	120	N/A	None	19.500 ns	ac[0]	t[7]
96	N/A	None	20.900 ns	ac[1]	t[7]	121	N/A	None	19.500 ns	ac[2]	t[7]
97	N/A	None	20.900 ns	a[7]	t[7]	122	N/A	None	19.500 ns	ac[0]	t[3]
98	N/A	None	20.900 ns	ac[1]	t[3]	123	N/A	None	19.500 ns	ac[2]	t[3]
99	N/A	None	20.900 ns	a[2]	t[3]	124	N/A	None	19.500 ns	ac[0]	t[1]
100	N/A	None	20.900 ns	ac[1]	t[1]	125	N/A	None	19.500 ns	ac[2]	t[1]

126	N/A	None	19.400 ns	ac[0]	t[2]
127	N/A	None	19.400 ns	ac[2]	t[2]
128	N/A	None	18.900 ns	b[5]	gf
129	N/A	None	18.800 ns	au_en	gf
130	N/A	None	16.800 ns	ac[1]	gf
131	N/A	None	15.900 ns	ac[0]	gf
132	N/A	None	15.900 ns	ac[2]	gf
133	N/A	None	15.100 ns	ac[3]	gf

结果分析及结论:

结果分析: Timing Analyzer Summary 显示, 最坏情况 tpd 为 29.400ns (b[2] 到 gf 的路径); tpd 报告表显示, 不同信号路径延时差异较大, 如 a[0] 到 gf 的延时为 27.900ns, ac[3] 到 t[0] 的延时为 21.900ns。

结论: AU 因包含加法器等复杂器件, 信号路径延时相对较大, 不同路径的固有延时不同, 实验中所有路径均满足时序要求, 无失败路径。

四、过程反思

1、知识与技能的掌握

应用 Verilog 语言的组合逻辑描述方法，实现了多路复用器、译码器和 AU 的功能设计，将理论知识转化为实际代码。

学会了 Quartus II 的工程创建、代码编译、仿真及时序分析等基本操作。

最具挑战性的是时序仿真中的延迟分析，通过查阅 Quartus 官方文档和参考实验指导书，理解了延时产生的原因，完成了时序结果解读。

2、能力发展

通过对比官方文档和实验案例，筛选有效信息，从逻辑功能和时序特性两个角度分析电路设计。

与同学讨论编译警告的解决方案（如引脚分配问题），通过分工验证代码，提高了沟通效率。尝试用 case 语句和条件判断两种方式编写多路复用器代码，最终选择逻辑更清晰的 case 语句实现。

3、反思与迭代

收到同学反馈代码中端口定义格式不统一，修改后提高了代码可读性；老师提示时序分析需关注最坏路径，补充了关键路径的延时解读。

3-2-1 反思：

3 件最重要的事：Verilog 组合电路的编写规范、Quartus 仿真与时序分析方法、组合电路的功能与时序验证逻辑。

2 个困惑 / 改进点：复杂电路的“冒险”现象消除方法、如何优化信号路径以缩短延时。

1 个感悟：硬件设计需兼顾逻辑正确性和时序稳定性，理论与实践结合才能真正掌握电路设计。

五、最终成果与自我评价

1、成果质量

达到预期：所有模块均通过功能仿真验证，编译无错误，时序分析无失败路径，实现了实验要求的全部功能。

改进方向：可优化代码逻辑减少信号路径长度，尝试消除时序仿真中的“冒险”现象，进一步提升时序性能。

2、自我评价

打分：90 分

理由：完整完成了 4 个模块的设计、仿真与分析，功能逻辑正确，符合实验要求；但在时序优化和“冒险”消除上还有提升空间，过程反思真实反映了实验中的学习与困惑。